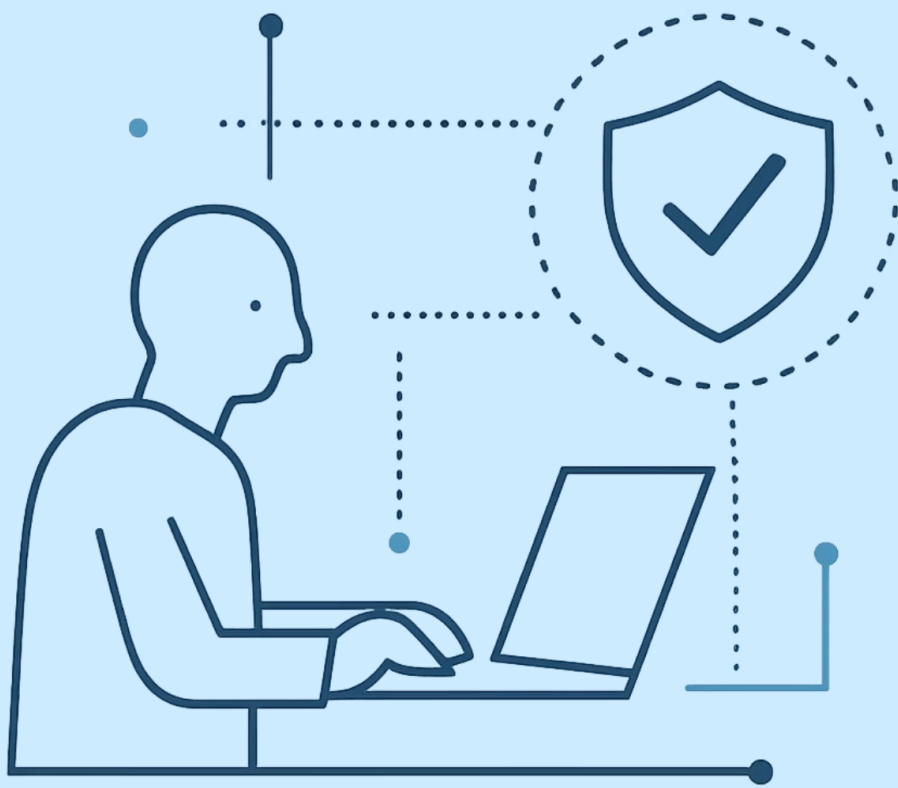


Angular Moderno

29 - Guards + Interceptors - Proteção de Rotas Admin



29 - Guards + Interceptors: Proteção de Rotas Admin

Implementar controle de acesso por perfil (usuário vs admin) utilizando Guards e Interceptors no Angular moderno, evoluindo a autenticação para um cenário mais próximo do mundo real.



Utilizar exclusivamente Angular moderno: Standalone, Signals, Guards funcionais e `HttpInterceptorFn`. NÃO utilizar `NgModule`, class-based interceptor ou conceitos legados.

Introdução da Aula

Contexto Atual do Projeto

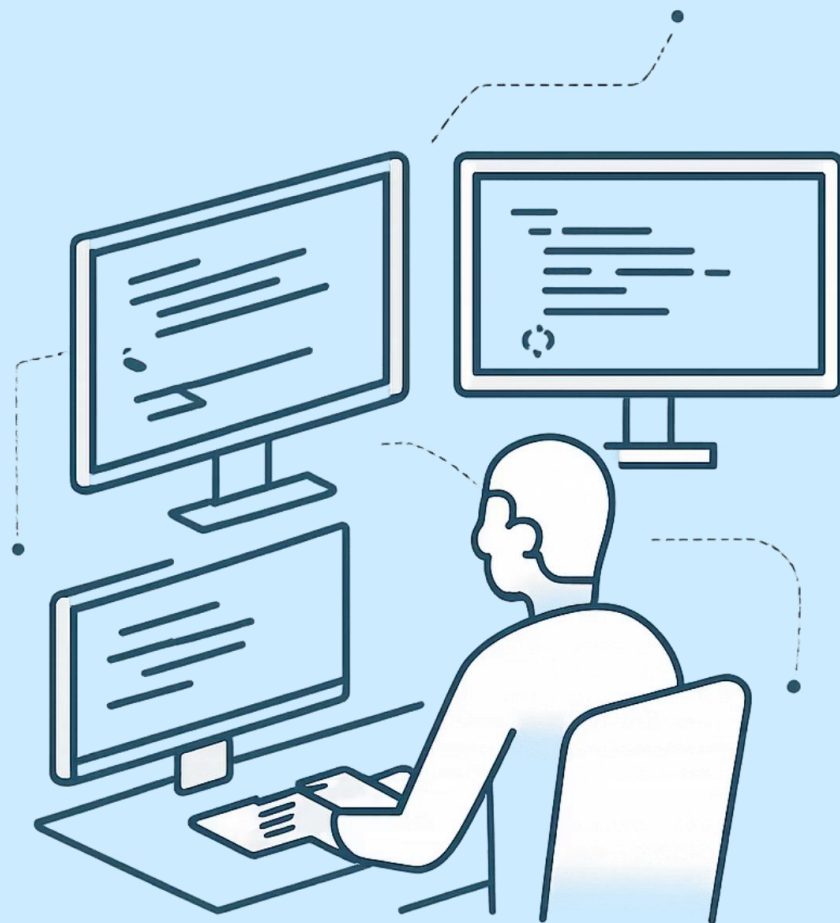
- Login com JWT simulado implementado
- AuthService com Signals
- Guard protegendo rotas autenticadas
- Interceptor já funcional

Problema Atual

- Todos os usuários possuem o mesmo nível de acesso
- Não existe separação entre usuário comum e administrador
- Não há controle de acesso por perfil

Objetivo desta Aula

- Criar perfil de usuário (usuario/admin)
- Proteger rota administrativa
- Ajustar fluxo de navegação
- Melhorar comportamento do interceptor



Conceito — Autorização vs Autenticação

Autenticação

Verifica se o usuário está **logado** no sistema.

Responde à pergunta: *"Quem é você?"*

Autorização

Verifica o que o usuário pode **acessar** dentro do sistema.

Responde à pergunta: *"O que você pode fazer?"*

authGuard

Responsável pela **autenticação**: Verifica se o usuário está logado antes de permitir acesso à rota.

adminGuard

Responsável pela **autorização**: Verifica se o usuário autenticado possui perfil de administrador.

Etapa 0 — Parar o Servidor

Ação Obrigatória

CTRL + C

Execute no terminal antes de qualquer alteração.

Por que é necessário?

- Evitar inconsistências durante a criação de arquivos
- Garantir compilação correta após as alterações
- Prevenir erros de hot-reload durante refatorações críticas

 Nunca pule esta etapa. Alterações em arquivos de rota e serviços com o servidor rodando podem gerar comportamentos inesperados difíceis de depurar.

Etapa 1 — Criar Feature Admin

Comando Angular CLI

```
ng generate component features/admin/admin
```

Caminho Criado

```
src/app/features/admin/admin/
```

Arquivos Gerados

admin.ts

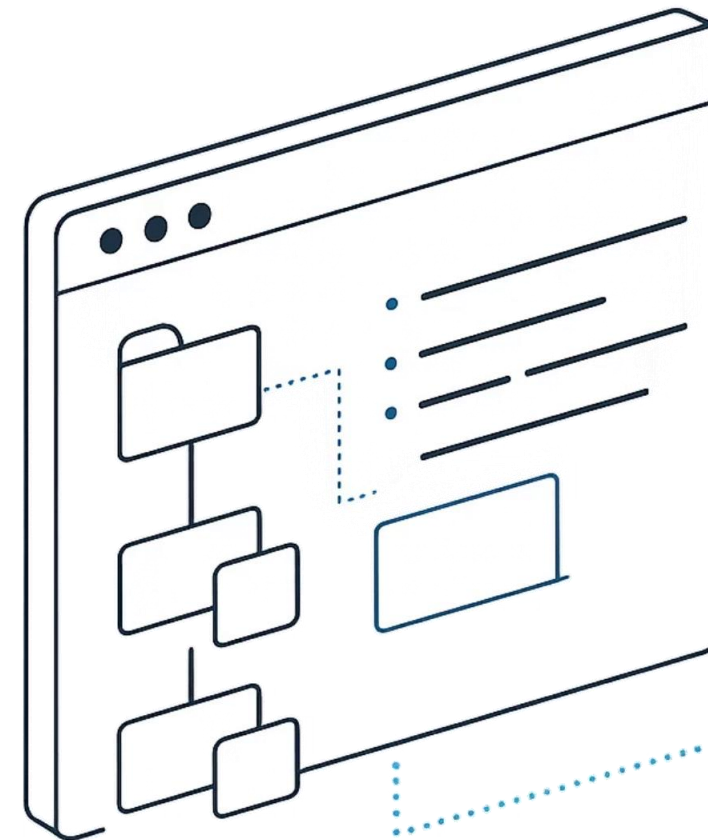
Componente principal

admin.html

Template da view

admin.css

Estilos do componente



Objetivo: Criar uma área administrativa isolada para aplicar proteção por perfil de acesso.

Etapa 2: Código — admin.ts

SRC/APP/FEATURES/ADMIN/ADMIN/ADMIN.TS

Responsável por representar o componente da área administrativa da aplicação.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-admin',
  imports: [],
  templateUrl: './admin.html',
  styleUrls: ['./admin.css'],
})
export class Admin {}
```

⚠ Apenas valide! O Código de `admin.ts` já deve estar assim.

Etapa 3: Código — admin.html

SRC/APP/FEATURES/ADMIN/ADMIN/ADMIN.HTML

Define a interface visual da área administrativa exibida ao usuário.

```
<h2>Área Administrativa</h2>

<p>Esta página representa uma área restrita da aplicação.</p>

<p>Somente usuários com perfil de administrador devem conseguir acessar esta rota.</p>
```

- ❏ O template é simples e demonstrativo. Em produção, esta área conteria funcionalidades reais de administração como CRUD de usuários, dashboards e configurações avançadas.

Etapa 4: Código — admin.css

[SRC/APP/FEATURES/ADMIN/ADMIN/ADMIN.CSS](#)

Aplica estilos básicos para organização visual da área administrativa.

```
h2 {  
  margin-top: 16px;  
}  
  
p {  
  max-width: 600px;  
}
```

Etapa 5: Criar admin.guard.ts

Arquivo a Criar

```
src/app/core/admin.guard.ts
```

Tipo de Guard

Guard funcional moderno — `CanActivateFn`

Este arquivo deve ser criado manualmente dentro da pasta core.

Responsabilidades do Guard

01

Verificar Login

Checar se o usuário está autenticado no sistema

02

Verificar Perfil Admin

Confirmar se o usuário possui perfil de administrador

03

Redirecionar

Enviar para login (não autenticado) ou acesso-negado (sem permissão)

Etapa 5: Código — admin.guard.ts

SRC/APP/CORE/ADMIN.GUARD.TS

Responsável por proteger a rota administrativa, permitindo acesso apenas para usuários com perfil admin.

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';

import { AuthService } from '../services/auth.service';

export const adminGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);

  // Primeiro verifica se o usuário está logado.
  // Se não estiver, envia para a tela de login.
  if (!authService.estaLogado()) {
    return router.createUrlTree(['/login']);
  }

  // Depois verifica se o usuário possui perfil de administrador.
  // Se estiver logado, mas não for admin, envia para produtos.
  if (!authService.ehAdmin()) {
    return router.createUrlTree(['/acesso-negado']);
  }

  // Se estiver logado e for admin, libera o acesso.
  return true;
};
```

✅ Este é o padrão moderno de Guard funcional do Angular utilizando createUrlTree para redirecionamento seguro.

Etapa 6 – Parte 1: Atualizar AuthService

SRC/APP/CORE/SERVICES/AUTH.SERVICE.TS

O sistema atual não diferencia usuário comum de administrador. A solução é criar um tipo de perfil utilizando TypeScript Union Types.

ANTES

```
perfil: 'usuario'
```

Perfil fixo, sem possibilidade de variação. Todos os usuários são tratados como comuns.

DEPOIS

```
type PerfilUsuario = 'usuario' | 'admin';
```

Union Type TypeScript que permite dois valores possíveis: `'usuario'` ou `'admin'`.

 Union Types são uma feature poderosa do TypeScript que garantem type safety — o compilador avisará se você tentar atribuir qualquer valor diferente de `'usuario'` ou `'admin'`.

 AGUARDE! Não faça nada por enquanto. Código completo deste arquivo está mais adiante.

Etapa 6 – Parte 2: Atualizar AuthService

SRC/APP/CORE/SERVICES/AUTH.SERVICE.TS

LÓGICA DE PERFIL NO LOGIN

Após definir o tipo, é necessário adicionar a lógica que determina qual perfil o usuário recebe ao fazer login. (Dentro da classe, antes do token)

```
// Lógica de determinação de perfil
const perfil: PerfilUsuario =
  email === 'admin@email.com' ? 'admin' : 'usuario';
```

Email: admin@email.com

Perfil atribuído: **'admin'**

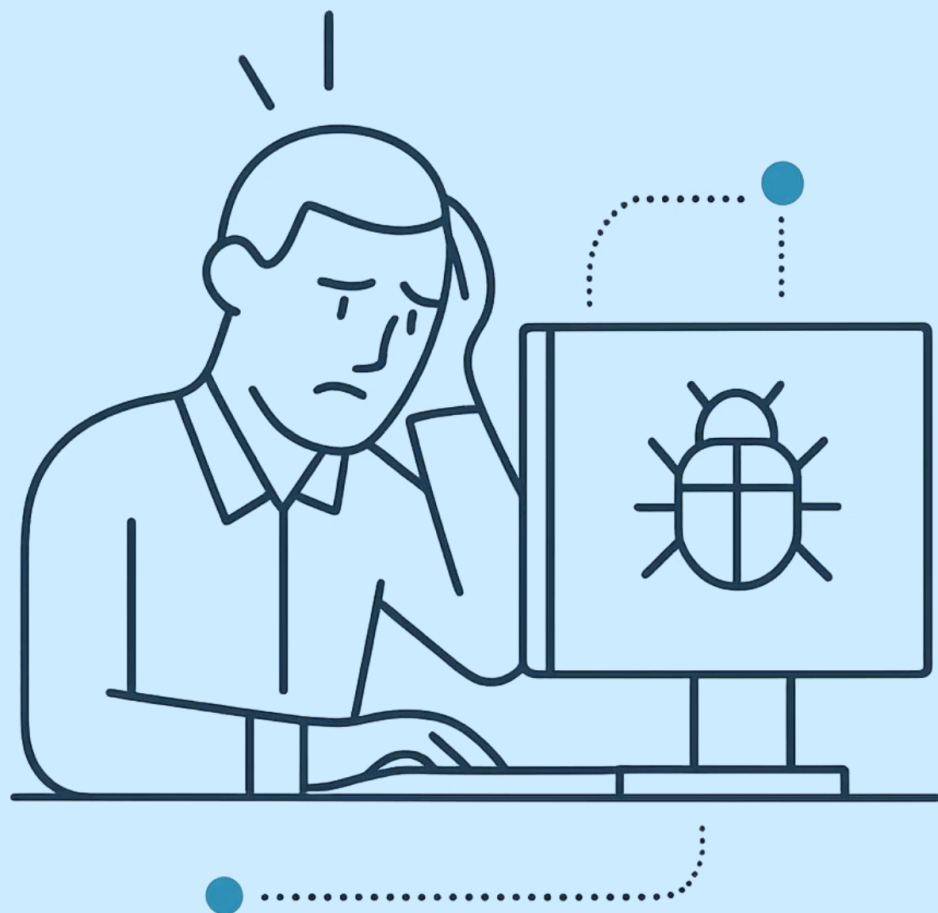
Terá acesso à rota **/admin** e ao botão no header.

Qualquer outro email

Perfil atribuído: **'usuario'**

Será redirecionado para **/produtos** após o login.

 AGUARDE! Não faça nada por enquanto. Código completo deste arquivo está mais adiante.



Etapa 6 – Parte 3: ERRO REAL de Implementação

SRC/APP/CORE/SERVICES/AUTH.SERVICE.TS

Código com Erro

```
this.usuario.set({  
  email,  
  perfil: 'usuario' // ERRADO!  
});
```

Problema

- Ignora completamente a variável `perfil` criada
- Sempre atribui `'usuario'` como perfil fixo
- Admin nunca consegue acessar a área administrativa
- Erro silencioso — não gera exceção no console

Correção

```
this.usuario.set({  
  email,  
  perfil // CORRETO!  
  // Shorthand property: usa  
  // o valor da variável perfil  
});
```

Por que funciona?

Em JavaScript/TypeScript, quando o nome da propriedade é igual ao nome da variável, podemos usar a sintaxe de **shorthand property**.

`{ perfil }` é equivalente a `{ perfil: perfil }`.

⚠ AGUARDE! Não faça nada por enquanto.
Código completo deste arquivo está mais adiante.

Etapa 6 – Parte 4: Atualizar AuthService

SRC/APP/CORE/SERVICES/AUTH.SERVICE.TS

COMPUTED SIGNAL PARA ADMIN

Com o perfil sendo armazenado corretamente no Signal, podemos criar um **computed** que deriva automaticamente se o usuário atual é administrador.

```
// Computed Signal derivado do estado do usuário  
ehAdmin = computed(() => this.usuario()?.perfil === 'admin');
```



Reatividade Automática

O `computed` recalcula automaticamente sempre que o Signal `usuario` muda — sem necessidade de subscriptions ou eventos manuais.



Derivação de Estado

É uma boa prática derivar estados secundários de estados primários. `ehAdmin` é sempre consistente com `usuario`.



Uso em Guards e Templates

O mesmo `computed` pode ser usado no `adminGuard`, no `header.html` e em qualquer outro lugar que precise verificar o perfil.

⚠️ AGUARDE! Não faça nada por enquanto. Código completo deste arquivo está mais adiante.

Etapa 6 – Agora sim! Código Completo – auth.service.ts

[SRC/APP/CORE/SERVICES/AUTH.SERVICE.TS](#)

```
import { Injectable, computed, signal } from '@angular/core';

type PerfilUsuario = 'usuario' | 'admin';

type Usuario = {
  email: string;
  perfil: PerfilUsuario;
};

@Injectable({
  providedIn: 'root',
})
export class AuthService {
  private usuario = signal<Usuario | null>(null);
  private tokenJwt = signal<string | null>(null);

  usuarioAtual = computed(() => this.usuario());
  estaLogado = computed(() => this.usuario() !== null);
  ehAdmin = computed(() => this.usuario()?.perfil === 'admin');
  token = computed(() => this.tokenJwt());

  login(email: string, senha: string): boolean {
    if (!email || !senha) {
      return false;
    }

    // Regra: e-mail perfil será admin: admin@email.com . Outro e-mail: usuario comum.
    const perfil: PerfilUsuario = email === 'admin@email.com' ? 'admin' : 'usuario';

    const tokenSimulado =
      'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.' +
      'eyJzdWIiOiJhbnVub0B0ZXN0ZS5jb20iLCJwZXI6ImFwbi0iLCJ1c2Vybm91dCI6ImF1dG8iLCJpYXN0IjoiYXN0aW50IiwiaWF0IjoiMTYxMjM0NTYiLCJ0b2tlbnR5cCI6IkpXVCJ9.' +
      'assinatura-simulada';
```

```
    this.usuario.set({
      email,
      perfil,
    });

    this.tokenJwt.set(tokenSimulado);

    return true;
  }

  logout() {
    this.usuario.set(null);
    this.tokenJwt.set(null);
  }

  obterToken(): string | null {
    return this.tokenJwt();
  }

  obterPerfil(): PerfilUsuario | null {
    return this.usuario()?.perfil ?? null;
  }
}
```

Etapa 7 — Atualizar Rotas

SRC/APP/APP.ROUTES.TS

Com o componente e o guard criados, é necessário registrar a rota administrativa no arquivo de rotas da aplicação.

```
// Adicionar ao array de rotas:  
{  
  path: 'admin',  
  canActivate: [adminGuard],  
  loadChildren: () => import('./features/admin/admin/admin').then((m) => m.Admin),  
},
```

path: 'admin'

Define a URL `/admin` como caminho da rota administrativa.

canActivate: [adminGuard]

Aplica o guard funcional que verifica autenticação e perfil antes de ativar a rota.

loadComponent (Lazy Loading)

Carrega o componente Admin apenas quando a rota é acessada, otimizando o bundle inicial da aplicação.

⚠ AGUARDE! Não faça nada por enquanto. Código completo deste arquivo está mais adiante.

Etapa 7: Código Completo — app.routes.ts

SRC/APP/APP.ROUTES.TS

```
import { Routes } from '@angular/router';
import { authGuard } from '../core/auth.guard';
import { adminGuard } from '../core/admin.guard';

export const routes: Routes = [
  {
    path: '',
    loadComponent: () => import('../features/home/home/home').then((m) => m.Home),
  },
  {
    path: 'produtos',
    loadComponent: () => import('../features/produtos/lista-produtos/lista-produtos').then((m) => m.ListaProdutos),
  },
  {
    path: 'carrinho',
    canActivate: [authGuard],
    loadComponent: () => import('../features/carrinho/carrinho/carrinho').then((m) => m.Carrinho),
  },
  {
    path: 'checkout',
    canActivate: [authGuard],
    loadComponent: () => import('../features/checkout/checkout/checkout').then((m) => m.Checkout),
  },
  {
    path: 'admin',
    canActivate: [adminGuard],
    loadComponent: () => import('../features/admin/admin/admin').then((m) => m.Admin),
  },
  {
    path: 'login',
    loadComponent: () => import('../features/login/login/login').then((m) => m.Login),
  },
  {
    path: '**',
    redirectTo: '',
  },
];
```

Etapa 8 – Atualizar Header

SRC/APP/SHARED/LAYOUT/HEADER/HEADER.TS

O Header deve reagir ao estado do usuário e exibir informações relevantes da sessão.

O que o Header utiliza:

```
estaLogado = this.authService.estaLogado;  
usuarioAtual = this.authService.usuarioAtual;
```

O botão "Admin" NÃO depende do Header para controle de acesso.

A segurança da aplicação é feita pelos Guards de rota.

Por que esta abordagem?

- O Header reage automaticamente aos Signals do AuthService
- Exibe o usuário logado
- Permite logout
- Atualiza a interface sem necessidade de subscriptions

⚠ AGUARDE! Não faça nada por enquanto. Código completo deste arquivo está mais adiante.

Etapa 8 - Código Completo — header.ts

SRC/APP/SHARED/LAYOUT/HEADER/HEADER.TS

```
import { Component, inject } from '@angular/core';
import { Router, RouterLink } from '@angular/router';
import { MatToolbarModule } from '@angular/material/toolbar';
import { MatButtonModule } from '@angular/material/button';

import { CarrinhoService } from '../../../core/services/carrinho.service';
import { AuthService } from '../../../core/services/auth.service';

@Component({
  selector: 'app-header',
  imports: [MatToolbarModule, MatButtonModule, RouterLink],
  templateUrl: './header.html',
  styleUrls: ['./header.css'],
})
export class Header {
  private carrinhoService = inject(CarrinhoService);
  private authService = inject(AuthService);
  private router = inject(Router);

  quantidade = this.carrinhoService.quantidade;
  estaLogado = this.authService.estaLogado;
  usuarioAtual = this.authService.usuarioAtual;

  sair() {
    this.authService.logout();
    this.router.navigateByUrl('/login');
  }
}
```

 O padrão moderno usa `inject()` ao invés de injeção via construtor. Ambos funcionam, mas `inject()` é mais conciso e alinhado com o estilo funcional do Angular moderno.

Etapa 9 — Atualizar o Template Header

SRC/APP/SHARED/LAYOUT/HEADER/HEADER.HTML

No template, utilizamos a nova sintaxe de controle de fluxo do Angular v17+ para exibir condicionalmente o link de administração.

```
<!-- Adicionar no header.html -->  
<button mat-button routerLink="/admin">Admin</button>
```

No template, optamos em deixar o botão Admin visível sempre para facilitar os testes e mostrar que a segurança do sistema não está na UX. Está na lógica de validação do usuário.

 AGUARDE! Não faça nada por enquanto. Código completo deste arquivo está mais adiante.

Etapa 9: Código Completo — header.html

SRC/APP/SHARED/LAYOUT/HEADER/HEADER.HTML

```
<mat-toolbar color="primary">
  <span>Minha Loja</span>

  <span class="flex-spacer"></span>

  <button mat-button routerLink="/produtos">Produtos</button>
  <button mat-button routerLink="/carrinho">Carrinho ({{ quantidade() }})</button>
  <button mat-button routerLink="/checkout">Checkout</button>
  <button mat-button routerLink="/admin">Admin</button>

  @if (estaLogado()) {
    <span>{{ usuarioAtual()?.email }}</span>
    <button mat-button type="button" (click)="sair()">Sair</button>
  } @else {
    <button mat-button routerLink="/login">Entrar</button>
  }
</mat-toolbar>
```

❏ O bloco `@else` é a nova sintaxe do Angular v17+. Substitui o padrão antigo de usar dois `*ngIf` com condições opostas.

Etapa 10 — Ajustar Login (ERRO REAL CRÍTICO)

SRC/APP/FEATURES/LOGIN/LOGIN/LOGIN.TS

Código com Erro

```
// Após login bem-sucedido:  
this.router.navigateByUrl('/produtos');
```

Problema

- Admin é sempre redirecionado para `/produtos`
- O fluxo ignora completamente o perfil do usuário
- Admin nunca consegue acessar `/admin` via login
- Erro lógico — não gera exceção, difícil de detectar

Impacto do Erro

Guard Inútil

O adminGuard existe mas nunca é testado via fluxo normal de login

UX Quebrada

Admin precisa navegar manualmente para `/admin` após o login

Lógica Inconsistente

O perfil é salvo corretamente mas ignorado na navegação



AGUARDE! Não faça nada por enquanto.
Código completo deste arquivo está mais adiante.

Etapa 10 – Parte 2: Correção do Login – Navegação por Perfil

LÓGICA DE REDIRECIONAMENTO

A correção consiste em verificar o perfil do usuário após o login e redirecionar para a rota correta de acordo com as permissões.

```
// Substituir a navegação simples por:  
if (this.authService.isAdmin()) {  
  this.router.navigateByUrl('/admin');  
  return;  
}  
  
// Usuário comum vai para produtos  
this.router.navigateByUrl('/produtos');
```

1

Login Realizado

AuthService processa credenciais e define perfil

2

Verificar Perfil

`isAdmin()` retorna true ou false

3

Redirecionar

Admin → `/admin` | Usuário → `/produtos`

⚠ AGUARDE! Não faça nada por enquanto. Código completo deste arquivo está mais adiante.

Etapa 10 - Código Completo — login.ts

SRC/APP/FEATURES/LOGIN/LOGIN/LOGIN.TS

```
import { Component, inject, signal } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router } from '@angular/router';

import { AuthService } from '../../../core/services/auth.service';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule],
  templateUrl: './login.html',
  styleUrls: ['./login.css'],
})
export class Login {
  private authService = inject(AuthService);
  private router = inject(Router);

  erroLogin = signal(false);

  formulario = new FormGroup({
    email: new FormControl('', [Validators.required, Validators.email]),
    senha: new FormControl('', [Validators.required, Validators.minLength(3)]),
  });
```

```
  entrar() {
    this.erroLogin.set(false);

    if (this.formulario.invalid) {
      this.formulario.markAllAsTouched();
      return;
    }

    const email = this.formulario.value.email ?? '';
    const senha = this.formulario.value.senha ?? '';

    const loginRealizado = this.authService.login(email, senha);

    if (!loginRealizado) {
      this.erroLogin.set(true);
      return;
    }

    if (this.authService.ehAdmin()) {
      this.router.navigateByUrl('/admin');
      return;
    }

    this.router.navigateByUrl('/produtos');
  }
}
```

Etapa 11 – Parte 1: Atualizar Interceptor

SRC/APP/CORE/INTERCEPTORS/HTTP.INTERCEPTOR.TS

O interceptor HTTP precisa tratar adequadamente os erros de autenticação e autorização retornados pela API.

401 – Unauthorized

Usuário não autenticado ou token expirado.

Ação: Executar logout e redirecionar para `/login`

403 – Forbidden

Usuário autenticado mas sem permissão para o recurso.

Ação: Redirecionar para `/produtos` sem logout

 A diferença entre 401 e 403 é fundamental: 401 significa "quem é você?" (não autenticado), enquanto 403 significa "eu sei quem você é, mas você não pode fazer isso" (não autorizado).

 **AGUARDE!** Não faça nada por enquanto. Código completo deste arquivo está mais adiante.

Etapa 11: Código Completo — http.interceptor.ts

SRC/APP/CORE/INTERCEPTORS/HTTP.INTERCEPTOR.TS

```
import { inject } from '@angular/core';
import { HttpInterceptorFn } from '@angular/common/http';
import { Router } from '@angular/router';
import { tap, catchError, throwError } from 'rxjs';

import { AuthService } from '../services/auth.service';

export const httpInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);

  // Router usado p redirecionamentos em erros de autenticação/autorização.
  const router = inject(Router);
  const token = authService.obterToken();

  console.log('REQUEST', req.url);
```

```
    const novaReq = token
      ? req.clone({
          headers: {
            Authorization: `Bearer ${token}`,
          },
        })
      : req;

    return next(novaReq).pipe(
      tap({
        next: (event) => console.log('RESPONSE:', event),
        error: (error) => console.error('ERRO:', error),
      }),
      catchError((error) => {
        console.error('ERRO GLOBAL:', error);

        // 401 -> ausência de autenticação ou token inválido.
        if (error.status === 401) {
          console.warn('Não autorizado. Faça login novamente.');
```

```
          authService.logout();
          router.navigateByUrl('/login');
        }

        // 403 -> usuario autenticado, mas sem permissão.
        if (error.status === 403) {
          console.warn('Acesso proibido. Perfil sem permissão.');
```

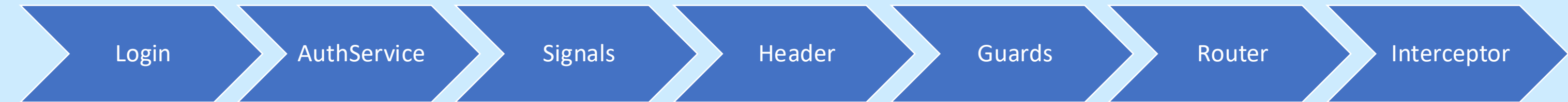
```
          router.navigateByUrl('/produtos');
        }

        // 500 -> erro interno do servidor
        if (error.status === 500) {
          console.warn('Erro interno do servidor!');
```

```
        }

        return throwError(() => error);
      })
    );
  };
};
```

Fluxo Completo da Aplicação



Cada camada da arquitetura tem uma responsabilidade clara e bem definida.

O estado flui de forma reativa a partir do AuthService, garantindo consistência em toda a aplicação.

O AuthService concentra o estado.

O Header reage ao estado.

Os Guards controlam o acesso às rotas.

O Login decide o redirecionamento inicial.

O Interceptor atua nas requisições HTTP.

Responsabilidades da Arquitetura



AuthService

Estado e Regras de Negócio

- Armazena o estado do usuário via Signals
- Define lógica de perfil (usuario/admin)
- Expõe computed Signals derivados



Guard

Controle de Acesso às Rotas

- Verifica autenticação antes de ativar rota
- Verifica perfil para rotas restritas
- Redireciona quando necessário



Login

Fluxo de Navegação Pós-Login

- Decide para onde redirecionar após autenticação
- Consulta perfil do AuthService
- Separa fluxo admin do fluxo usuário



Header

Interface Reativa ao Estado

- Exibe elementos condicionalmente
- Reage automaticamente a mudanças de Signal
- Sem lógica de negócio própria



Interceptor

Comunicação HTTP Centralizada

- Injeta token em todas as requisições
- Trata erros 401 e 403 globalmente
- Evita repetição de código nos serviços

Testes Obrigatórios

VALIDAÇÃO DA IMPLEMENTAÇÃO

Após implementar todas as etapas, execute os seguintes testes para garantir que tudo está funcionando corretamente:

1

Acesso sem Login

Acessar `/admin` sem estar autenticado → deve redirecionar para `/login`

2

Login como Usuário

Fazer login com qualquer email diferente de `admin@email.com` → deve redirecionar para `/produtos`

3

Login como Admin

Fazer login com `admin@email.com` → deve redirecionar para `/admin` e exibir a área administrativa

4

Header Reativo

Verificar que o botão "Admin" permanece visível para todos, mas o acesso real é controlado pelo adminGuard: usuário comum deve ir para `/acesso-negado` e admin deve acessar `/admin`.

5

Interceptor Funcional

Confirmar no console que o Interceptor continua registrando REQUEST, RESPONSE e tratando erros HTTP globais.

Conclusão da Aula

Autorização por Perfil

Implementamos diferenciação entre usuário comum e administrador usando Union Types e Signals.

Guard Especializado

Criamos o `adminGuard` funcional que verifica autenticação e perfil antes de ativar rotas restritas.

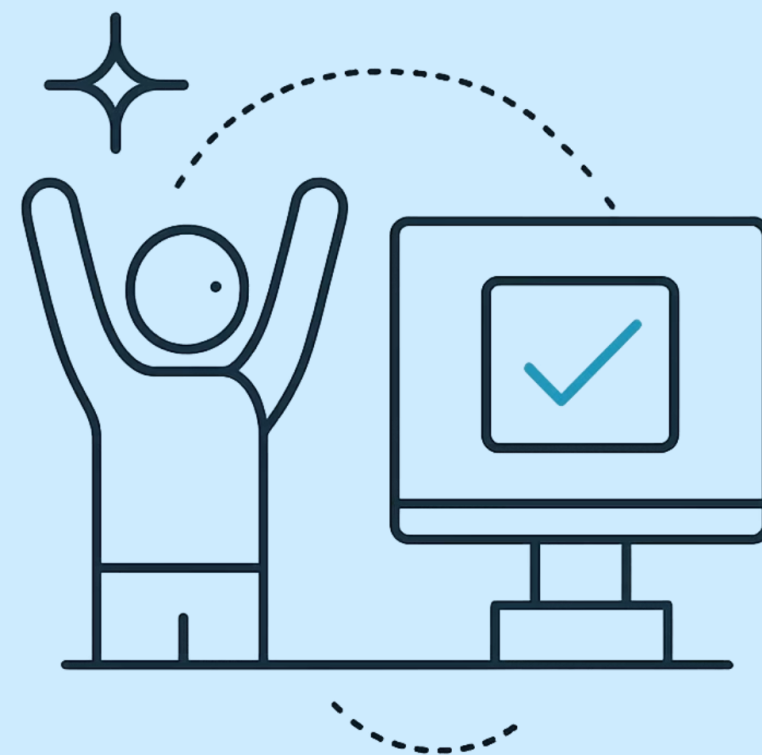
AuthService Evoluído

Adicionamos tipo de perfil, lógica de determinação e computed Signal `ehAdmin`.

Erros Reais Corrigidos

Identificamos e corrigimos dois erros críticos: perfil fixo no `set()` e navegação ignorando perfil.

Mensagem Final: Separar autenticação, autorização e navegação é essencial para aplicações escaláveis. Cada camada deve ter uma única responsabilidade bem definida. Este é o caminho para um sistema profissional e manutenível.



Dúvidas

?

